MACAU UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Computer Science and Engineering

Faculty of Innovation Engineering

Final Year Project Progress Report

Title: AI-Assisted Teaching System for Programming Courses — Backend & Database Focus

Student Name : Liu Shiyu

Student No. : 1220032841

Supervisor : Tian Jinyu

October, 2025

Abstract

This report summarizes the progress of developing the backend of an AI-assisted programming course teaching system. The system aims to improve the efficiency of grading and the quality of feedback in programming courses by combining program analysis, sandboxed execution, and AI-generated feedback. The core goal of the project is to build an automated grading and feedback system that generates syllabus-aligned feedback and evaluates the correctness, quality, and efficiency of student code. Currently, the implementation of API routes for user login, code submission, and analytics has been completed, and the database schema has been set up. The feedback generation system uses Retrieval-Augmented Generation (RAG) technology to provide personalized feedback to students based on course materials. Preliminary tests have successfully implemented code submission and grading functionalities. The next steps include further refining the RAG pipeline, finalizing the grading rubric, and conducting full system integration and testing. This report details the current progress and the plans for the next phase of the project.

Table of Contents

Abstract 1
1. Introduction 3
2. Objectives 5
3. Related work 7
4. Methodology 9
5. Preliminary experimental results 11
6. Project planning 14
References 15

1. Introduction

As the scale of programming courses continues to expand, a significant challenge faced by instructors is how to efficiently and consistently provide timely feedback to a large number of students. Traditional manual grading is not only time-consuming but also prone to inconsistencies in grading standards and delayed feedback, especially in large-scale courses. Although some automatic grading systems (e.g., unit test-based autograders) provide certain levels of support, they typically focus only on functional correctness and overlook other important aspects, such as code quality, design, and documentation. Moreover, these systems often fail to provide deep, explanatory feedback that guides students on how to improve.

There are also situations requiring discretionary handling, such as when a student's code contains errors preventing direct compilation, yet the underlying logic and functions remain correct. Most traditional automated grading models would award zero marks outright in such cases. However, in the eyes of the majority of tutors, such code could well be considered a valid solution. Awarding zero marks solely due to minor oversights or formatting errors does not align with contemporary teaching practices. This project aims to address precisely such challenges.

With advancements in large language models (LLMs) for code understanding and generation, AI technology has begun to gain attention in the education field. Models such as CodeBERT and CodeT5 have made significant progress in code generation and understanding, enabling richer feedback and suggestions. However, the application of these models in education still faces challenges, such as the generation of content that may not align with the course syllabus, suggestions that go beyond the students' current learning scope, and issues with feedback accuracy and consistency.

To address these issues, this project proposes an AI-assisted programming course teaching system focused on the backend. The system combines program analysis, sandbox execution, and Retrieval-Augmented Generation (RAG) technology to provide personalized, syllabus-aligned feedback through an automated grading and feedback generation mechanism. The system design

considers academic integrity, privacy protection, and security, ensuring that the feedback only covers the topics students have already learned, avoiding suggestions that are out of scope.

The core functionalities of the system include automatic grading of code submissions, feedback generation, and assignment management. It integrates various course materials, such as lecture notes, code examples, and grading rubrics, with an AI feedback generation model. This will generate in-depth feedback with suggestions for improvement specific to each student's submission. The database architecture will ensure the persistent storage of all data, including submission records, grading rubrics, feedback, and execution logs, supporting further analysis and reviews. Through this approach, the project aims to provide a scalable, efficient, and consistent grading and feedback mechanism for programming courses, alleviating the burden on instructors and enhancing the quality of education.

2. Objectives

The objective of this project is to develop the backend of an AI-assisted programming course teaching system that aims to improve the efficiency of grading and the quality of feedback in programming courses. The specific objectives of the project are as follows:

1. **Design and implement an efficient backend service**
   The primary function of the backend service is to receive student code submissions, execute them in a sandboxed environment, and generate grades and feedback based on predefined grading rubrics. The system must be capable of handling a large number of submissions automatically while ensuring the accuracy and consistency of feedback. The backend will provide APIs for integration with other learning platforms, supporting real-time submission, grading, and feedback retrieval. Additionally, the system will handle code execution and analysis in various programming languages, ensuring compatibility with multiple course requirements.

2. **Implement a flexible and configurable grading rubric**
   To ensure fairness and transparency in the grading process, the project will design a highly configurable grading rubric. The rubric will evaluate student code based on multiple factors, such as functional correctness, code quality, efficiency, and documentation. Each grading factor will be assigned a weight based on the course requirements, ensuring that different types of programming tasks are assessed appropriately. The system will support version control for grading rubrics, ensuring that historical grading criteria can be tracked and adjusted based on changes to the course content.

3. **Build a concept-aware constraint mechanism**
   This project will implement a concept-aware constraint mechanism to ensure that the feedback generated is consistent with the student's current learning progress and course syllabus. For example, the system will automatically control the feedback content based on the course schedule, ensuring that students do not receive inappropriate suggestions or

prompts about topics they have not yet studied. This mechanism will dynamically adjust feedback to avoid suggestions of techniques or tools that are outside the scope of the course, ensuring that feedback aligns with the student's learning stage.

4. **Implement a Retrieval-Augmented Generation (RAG) feedback pipeline**
   The project will build a RAG pipeline to enhance the generated feedback by retrieving relevant course materials (e.g., syllabi, lecture notes, code examples) from course-related documents. RAG technology can retrieve pertinent information from course materials and use it to inform the feedback generation. This approach will effectively prevent AI-generated feedback from deviating from course content and improve its relevance and practical value. The system will support the generation of personalized feedback for each student, based on both the course content and the specifics of their assignment.

5. **Design and implement a secure, auditable database system**
   The project will utilize a relational database system with strong security measures to ensure the privacy and integrity of student data. The database will store submission records, execution logs, grading rubrics, grading results, feedback content, and historical data. It will support comprehensive analysis of student submissions, grading, and feedback, allowing instructors to track common mistakes and student progress. Additionally, the database will include version control, ensuring that historical versions of course materials and grading rubrics are traceable and auditable.

6. **Ensure system reliability, fairness, and academic integrity**
   The system will implement measures to ensure fairness and academic integrity, including code similarity detection to prevent plagiarism and cheating. All grading and feedback processes will be logged with detailed tracking, ensuring that the system's behavior is auditable. The system will undergo regular fairness evaluations, comparing AI-generated grades with human graders to ensure accuracy and consistency.

7. **Conduct experimental validation and performance evaluation**
   In the later stages of the project, the system will be validated experimentally. The accuracy of AI-generated grades will be compared to human grading, and the usefulness of the feedback will be evaluated. Additionally, the system's performance will be assessed through stress testing and response time testing to ensure it can handle high concurrency and grading demands in real-world use.

The ultimate goal of this project is to create a grading system that provides automated, intelligent, and personalized feedback for programming courses, thereby enhancing teaching efficiency, reducing the burden on instructors, and offering students more precise and targeted learning guidance.

3. Related work

With the increasing demand for programming education, many research studies and tools have been developed to automate grading and provide feedback. However, existing automatic grading systems often focus only on functional correctness and lack assessment of other important aspects, such as code quality, design, and documentation. This section reviews the related work in automatic grading systems, the application of large language models (LLMs) in code analysis, Retrieval-Augmented Generation (RAG) technology, and academic integrity detection.

1. **Automatic Grading Systems**

   Traditional automatic grading systems typically rely on unit tests to evaluate the functional correctness of student code. While these systems can reduce the workload of instructors to some extent, they have significant limitations. Ihantola et al. (2010) reviewed existing automatic grading systems and pointed out that these systems fail to assess code design, readability, and conceptual understanding. For example, some systems only provide binary feedback (correct or incorrect), lacking deep diagnostics and guidance. Although automatic grading systems are widely used in large-scale courses, they often fail to meet the need for high-quality feedback in teaching.

2. **Application of Large Language Models in Code Understanding**

   In recent years, large language models (LLMs) have shown significant success in natural language processing and code generation, prompting their application in programming education. Models such as CodeBERT (Feng et al., 2020) and CodeT5 (Wang et al., 2021) excel in understanding and generating code. Chen et al. (2021) demonstrated how large language models can solve programming tasks through reasoning and code generation, while also being able to explain code errors. These models can generate more detailed feedback compared to traditional autograders, including code improvement suggestions, design refactoring, and additional tests. However, the use of LLMs in education still faces challenges. For instance, LLMs may generate suggestions that are inconsistent with the content students have learned, or provide solutions that are beyond their current level of understanding. Additionally, LLMs can suffer from hallucinations, where they generate inaccurate or irrelevant feedback.

3. **Retrieval-Augmented Generation (RAG) Technology**

   To address the issue of LLMs generating content beyond the course syllabus, Retrieval-Augmented Generation (RAG) technology has emerged. RAG combines retrieval with generation to enhance the output of language models by retrieving relevant information from external documents. Lewis et al. (2020) proposed the RAG framework, which effectively aligns model outputs with specific contexts by retrieving pertinent

information. In educational applications, RAG can enhance feedback generation by retrieving course materials (e.g., syllabi, lecture notes, code examples) and using them to inform the feedback. This approach minimizes the risk of AI-generated feedback deviating from course content, ensuring that the feedback is more specific, relevant, and actionable. Compared to traditional LLMs, RAG ensures that feedback remains closely tied to course content, thus improving its relevance and practical value.

4. **Academic Integrity Detection**

   Academic integrity is a critical issue in programming education, especially in large-scale courses where plagiarism and collusion among students can affect grading fairness. Existing academic integrity detection tools, such as JPlag, detect code similarities to identify potential plagiarism (Prechelt et al., 2002). These tools can help instructors identify cheating behaviors, but they often lack a comprehensive analysis of generated code and cannot provide real-time feedback. Therefore, combining LLMs and RAG in automatic grading systems not only enables plagiarism detection but also provides personalized feedback based on course content, improving grading fairness and teaching effectiveness.

5. **Combining Feedback and Grading Rubrics**

   Many studies emphasize the importance of feedback and grading rubrics in programming education. Brookhart (2013) discussed how grading rubrics can enhance fairness and transparency in grading. Rubrics not only clarify how students will be assessed across various dimensions but also provide clear guidance for improvement. In programming education, rubrics typically cover functional correctness, code quality, documentation, and testing. However, traditional grading rubrics are often manually created and are not flexible enough to accommodate different courses and tasks. Therefore, combining automatic grading systems with flexible rubrics can provide more transparent and fair assessments for students.

In summary, existing automatic grading systems, LLMs, and RAG technology all have significant potential in programming education but still face several challenges. By combining these technologies, this project aims to develop an automated grading system that provides high-quality feedback, aligns with the course syllabus, and emphasizes academic integrity, offering a more intelligent and efficient solution for programming education.

4. Methodology

The objective of this project is to develop the backend of an AI-assisted programming course teaching system that aims to improve the efficiency of grading and the quality of feedback in programming courses. The following describes the methodologies and design principles that we have employed to achieve this goal.

1. Fully Implemented Methods and Designs

- **Backend Framework**
  We chose FastAPI as the backend development framework due to its high performance, asynchronous support, and scalability. This framework enables us to handle a large number of concurrent requests, meeting the growing demands of large-scale courses. The API routes include modules for user login, assignment submission, grading, and feedback retrieval. Using FastAPI's dependency injection (DI) feature, we have modularized the code, allowing each function to be developed and tested independently, making future maintenance and expansion easier.

- **Database Architecture**
  We have adopted PostgreSQL as the relational database for the system to ensure consistency and integrity of the data. The database design takes into account the storage and management of multidimensional data, including student assignments, submission records, grading rubrics, and feedback content. To support the traceability of the grading process, detailed execution logs are stored for each assignment submission and grading result, ensuring that all data can be accurately audited and queried.

Partially Implemented Methods and Designs

- **Sandbox Execution Environment**
  Currently, student code submissions can be executed within a secure sandbox environment, with resource usage (e.g., memory, CPU time) being restricted to prevent potential security issues. The sandbox execution environment logs the results of each code execution, including whether unit tests pass, runtime, and memory usage. Based on these results, preliminary grades and feedback are generated.

- **Concept-Aware Constraint Mechanism**
  During the grading and feedback generation process, we have designed a concept-aware constraint mechanism to ensure that the generated feedback is aligned with the student's current learning progress. By dynamically adjusting feedback content according to the course syllabus, the system prevents generating suggestions that are beyond the student's scope. Although this mechanism has been partially implemented and integrated into the

feedback generation flow, its adaptability to different course contents still needs to be fully tested.

- **Integration of RAG Pipeline**
  The Retrieval-Augmented Generation (RAG) technology is a key part of the feedback generation process. The current implementation allows the retrieval of course materials and the generation of related feedback, but there is still room for improvement, especially in terms of enhancing the accuracy and specificity of the feedback. We plan to optimize the RAG pipeline further to ensure it can generate more personalized and context-specific feedback based on student submissions.

Future Design and Implementation

- **Enhancing the RAG Pipeline**
  We plan to optimize the integration of RAG technology to ensure more accurate feedback. Specifically, we will improve the efficiency and accuracy of the RAG pipeline by better processing course documents (e.g., syllabi, lecture notes, grading rubrics) and integrating them into the feedback generation process. We will also adjust the input and output of the RAG system based on the specific tasks and programming languages of student submissions.

- **Comprehensive Integration of Concept-Aware Constraints**
  In the future, we will further refine the concept-aware constraint mechanism, ensuring it tightly integrates with the course syllabus. The system will dynamically adjust the feedback content based on the student's learning progress, preventing the suggestion of concepts that have not been taught yet. This mechanism will be fully deployed in the next phase, ensuring that feedback remains accurate and relevant.

- **Improving the Database and Feedback Association**
  After completing the integration of the system's core features, we plan to further optimize the database design by tightly linking feedback content with student submission records, grading rubrics, and test results. Through data analysis, we will identify common learning issues, concept mastery, and other valuable insights to support teaching decisions.

Methodology Summary

This project employs a modular development approach with continuous iteration and optimization. By using modern frameworks and technologies such as FastAPI, PostgreSQL, and RAG, we have ensured the system's scalability and flexibility through sound system architecture, data storage, and feedback generation mechanisms. In the future, we will continue to test and refine each module, gradually realizing the complete functionality of the AI-assisted programming teaching system.

5. Preliminary experimental results

In this chapter, we present the comprehensive preliminary experimental results of the backend system. The scope of these experiments was strictly limited to the server-side components, including the API gateway, business logic layer, and data persistence layer. All interactions were performed programmatically using API testing tools (Postman/cURL) to simulate client requests, ensuring a focused evaluation of the backend's performance and reliability independent of any frontend implementation.

5.1 Experimental Environment and Infrastructure

The experiments were conducted in a containerized environment to ensure consistency and reproducibility.

- **Host Configuration**: The system was deployed on a local workstation running Windows 11, utilizing the Windows Subsystem for Linux (WSL 2) to host the Docker engine. This setup provides a Linux-compatible kernel interface essential for container operations.
- **Container Orchestration**: We employed **Docker Compose** to define and manage the multi-container application. The architecture consists of two primary services:
    1. **backend Service**: A Python-based container running the FastAPI application server (Uvicorn). It is configured with port mapping (8000:8000) to allow external API access.
    2. **db Service**: A PostgreSQL 16 database container. It is customized with the pgvector extension to support future vector similarity search features. It utilizes a named volume (postgres_data) to ensure data persistence across container restarts.

Verification of Service State:
Upon executing the startup command, we verified the service health through the Docker CLI.

- **Database Readiness**: The PostgreSQL logs confirmed that the database system was ready to accept connections within 15 seconds of initialization.
- **Migration Execution**: We verified that the Alembic migration script successfully ran and created the required tables in the database.

5.2 Functional Tests

5.2.1 User Authentication: Sign-Up and Login

We tested the user authentication process by simulating a sign-up and login flow.

- **Sign-Up Test**:
    - **Method**: A POST request was sent to /api/v1/users/ with a JSON payload containing user credentials (e.g., email: student_test@example.com and plain text password).
    - **Result**: The API returned a 200 OK status code, indicating successful resource creation.
    - **Security Validation**: We executed a direct SQL query on the users table and confirmed that the password field contained a **Bcrypt hash** string, not the plaintext password. This verified that the password hashing mechanism was working as designed to protect user credentials.
- **Token Issuance Test**:
    - **Method**: We sent a POST request to /api/v1/login/access-token with the sign-up credentials.
    - **Result**: The server responded with a JSON object containing the access_token and token_type: bearer.
    - **Token Validation**: We decoded the received JWT and verified it contained the correct sub (subject ID) and role claims. This confirmed that the authentication logic correctly identified the user and issued a valid session token.

5.2.2 Core Business Logic: Submission System

The main functionality of the backend is to handle code submissions. We tested the flow of data from API reception to database storage.

- **Submission Persistence Test**:
    - **Method**: We simulated a student submission by sending a POST request to /api/v1/submissions/. The request header contained the Bearer Token obtained from the login step, and the body contained a Python code snippet: print("Hello Backend").
    - **Result**: The API returned a 200 OK response along with a unique submission ID.
    - **Data Integrity Verification**: We queried the submissions table in the database using the returned ID. We compared the stored code_content with the original payload and found them to be identical. This proved that the backend correctly parsed, validated, and persisted the user data.

5.3 Reliability and Fault Analysis

While static data operations (CRUD) were successful, we encountered specific failure modes in the dynamic code execution component.

- **Experiment: Code Execution Sandbox**:
  We attempted to trigger the automatic grading logic, which requires the backend to create a temporary Docker container ("sandbox") to safely execute user code.
- **Observed Faults**:
  The API request resulted in a 500 Internal Server Error.
    - **Log Analysis**: Application logs displayed a docker.errors.DockerException. The error message indicated that communication with the Docker daemon socket (/var/run/docker.sock) failed.
- **Root Cause Analysis (RCA)**:
  The failure was attributed to a **Docker-in-Docker (DinD)** configuration issue.

1. **Socket Mapping Issue**: The backend application runs inside a Docker container. To create *sibling containers* (sandbox), it mounted the host's Docker socket. However, due to low-level differences between the Windows host file system and the Linux container environment, the socket path mapping failed to resolve correctly.

2. **Volume Mount Context**: The backend attempted to mount code files from its local path (/tmp/...) into the sandbox container. In sibling container setups, volume paths are interpreted relative to the *host*, not the invoking container. Since the host did not have the file at the specific path, the mount failed.

6. Project planning

Requirements Analysis and Architecture Design
This phase has been completed, including discussions with the supervisor and team members to clarify the project goals, functional modules, and technical architecture. We selected the FastAPI framework for backend development, PostgreSQL for database design, and Docker for containerization of code execution and system deployment.

Backend Development and Database Design
Backend development is partially completed, and the database architecture has been designed and implemented. Core functionalities such as user authentication, assignment submission, and grading feedback have undergone initial testing. Currently, we are continuing to optimize and integrate the API routes, which is expected to be completed next month.

Feedback Generation and Grading Rubric Integration
The feedback generation system, which uses Retrieval-Augmented Generation (RAG) technology, is able to extract relevant information from course materials and generate preliminary personalized feedback. This feature still requires further optimization to ensure that the generated feedback is more specific, accurate, and aligned with the course syllabus.

System Testing and Optimization
Initial functionality testing has been completed, but issues have been encountered in the code execution sandbox. These need further fixes. The next phase will focus on addressing Docker and sandbox compatibility issues to ensure system stability and high-concurrency handling.

Deployment and Delivery
After completing all development and testing, the system will enter the deployment phase. We will use Docker Compose for multi-container management and deploy it to the production environment.

References

[1] Andrade, H. G. (2005). Teaching with rubrics: The good, the bad, and the ugly. College teaching, 53(1), 27-31.

[2] Brookhart, S. M. (2013). How to create and use rubrics for formative assessment and grading. Ascd.

[3] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., ... & Zaremba, W. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374. [4] Cohen, J. (1960). A coefficient of agreement for nominal scales. Educational and psychological measurement, 20(1), 37-46.

[5] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., ... & Zhou, M. (2020). Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155. [6] Ihantola, P., Ahoniemi, T., Karavirta, V., & Seppälä, O. (2010, October). Review of recent systems for automatic assessment of programming assignments. In Proceedings of the 10th Koli calling international conference on computing education research (pp. 86-93).

[7] Jia, Y., & Harman, M. (2010). An analysis and survey of the development of mutation testing. IEEE transactions on software engineering, 37(5), 649-678.

[8] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535-547.

[9] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems, 33, 9459-9474.

[10] Prechelt, L., Malpohl, G., & Philippsen, M. (2002). Finding plagiarisms among a set of programs with JPlag. J. Univers. Comput. Sci., 8(11), 1016.

[11] Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., ... & Ma, S. (2020). Codebleu: a method for automatic evaluation of code synthesis. arXiv preprint arXiv:2009.10297.

[12] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., ... & Synnaeve, G. (2023). Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950.

[13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.